

Supplementary Materials: LLM-Assisted Extraction of QSP Calibration Targets

Joel Eliason

Contents

1	S1: Complete SubmodelTarget Example	2
1.1	Overview	2
1.2	Complete YAML Specification	2
1.3	Schema Structure Summary	2
2	S2: Supported Model Types	2
3	S3: Validation Checks	3
3.1	Summary of Validators	3
3.2	DOI Resolution Validator	3
3.3	Value-in-Snippet Validator	4
3.4	Unit Validation	4
3.5	Reference Consistency Validators	4
4	S4: Generated Julia Code Structure	4
5	S5: Pydantic Model Definitions	5
5.1	Top-Level Model	5
5.2	Input Model	5
5.3	Model Type Discriminated Union	5
5.4	Prior Specification	5
5.5	Loading and Validating YAML Files	5

1 S1: Complete SubmodelTarget Example

This section presents a complete calibration target for activated pancreatic stellate cell (aPSC) proliferation, demonstrating all schema components.

1.1 Overview

This target constrains the proliferation rate parameter `k_apsc_prolif` using data from Schneider et al. (2001), who measured PSC proliferation in response to PDGF stimulation via BrdU incorporation assay.

1.2 Complete YAML Specification

```
# Schema: SubmodelTarget
# Target: Activated PSC proliferation rate

target_id: psc_proliferation_PDAC_deriv001

# =====
# INPUTS: Extracted values with full provenance
# =====
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                    0.39-fold of control)."
```

```
  - name: pdgf_fold_sd
    value: 0.49
    units: dimensionless
    input_type: direct_measurement
    role: auxiliary
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                    0.39-fold of control)."
```

```
# =====
# CALIBRATION: Everything needed for inference
# =====
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      # log(1.0) - expect ~1/day for 4x fold-change in 1 day
        sigma: 1.0
        rationale: |
          Wide log-normal prior centered at 1/day. A 4-fold increase in
          1 day implies  $k = \ln(4) = 1.4/\text{day}$ , so prior median of 1/day
          is reasonable.
```

```

state_variables:
- name: aPSC_rel
  units: dimensionless
  initial_condition:
    value: 1.0
    rationale: |
      PSC proliferation data are reported as fold of control. We
      normalize the initial aPSC population to 1.0 (100% control)
      to match this convention.

model:
  type: exponential_growth
  rate_constant: k_apsc_prolif
  data_rationale: |
    BrdU incorporation assay measures proliferation as fold-change
    over ~24 hours under constant PDGF stimulus. With a single high
    PDGF dose and short duration, exponential growth ( $dN/dt = k \cdot N$ )
    fits the data structure.
  submodel_rationale: |
    The full QSPIO_PDAC model has aPSC proliferation via:
     $k_{apsc\_prolif} * aPSC * (PDGF / (PDGF_{50\_prolif} + PDGF))$ .
    At 50 ng/mL PDGF  $\gg$   $PDGF_{50\_prolif}$ , the saturation term  $\sim 1$ ,
    reducing to exponential growth with rate  $k_{apsc\_prolif}$ .

independent_variable:
  name: time
  units: day
  span: [0.0, 1.0]
  rationale: |
    Proliferation assay duration assumed to be ~24 hours based on
    typical cytokine incubation protocols for rat PSCs.

measurements:
- name: proliferation_fold_change
  observable:
    type: identity
    state_variables: [aPSC_rel]
    rationale: |
      The state variable aPSC_rel is the fold-change in aPSC number
      relative to baseline (control = 1.0), which directly matches
      the reported proliferation fold-change.
  units: dimensionless
  uses_inputs:
    - pdgf_fold_mean
    - pdgf_fold_sd
  evaluation_points: [1.0]
  sample_size: 3
  sample_size_rationale: |
    Sample size not explicitly reported; assumed n=3 independent
    experiments, typical for in vitro PSC BrdU proliferation assays.
  distribution_code: |
    def derive_distribution(inputs, ureg):
        import numpy as np

        rng = np.random.default_rng(42)

        mean = inputs['pdgf_fold_mean'].magnitude
        sd = inputs['pdgf_fold_sd'].magnitude
        n = 10000

```

```

    # Use lognormal for positive fold-changes
    mu_log = np.log(mean**2 / np.sqrt(mean**2 + sd**2))
    sigma_log = np.sqrt(np.log(1 + sd**2 / mean**2))

    samples = rng.lognormal(mu_log, sigma_log, n)

    median_val = np.median(samples)
    ci_lower = np.percentile(samples, 2.5)
    ci_upper = np.percentile(samples, 97.5)

    return {
        'median': [median_val],
        'ci95': [[ci_lower, ci_upper]],
        'units': 'dimensionless'
    }
likelihood:
    distribution: lognormal
    rationale: |
        Fold-changes are positive and multiplicative, making
        lognormal appropriate.

    identifiability_notes: |
        With a single timepoint (~24h) and normalized initial condition
        (aPSC_rel(0)=1), only k_apsc_prolif is estimated. If substantial
        PSC death occurred over the assay interval, the inferred rate
        would represent net growth rather than pure proliferation.

# =====
# EXPERIMENTAL CONTEXT
# =====
experimental_context:
    species: rat
    system: in_vitro_primary_cells
    indication: PDAC
    cell_types:
        - name: pancreatic stellate cell
          phenotype: activated (alpha-SMA positive)
          isolation_method: Isolated from normal Wistar rat pancreas,
                           activated by culture on plastic
    culture_conditions:
        culture_type: 2d_monolayer

# =====
# STUDY INTERPRETATION
# =====
study_interpretation: |
    Schneider et al. (2001) measured proliferation of cultured rat
    pancreatic stellate cells (PSCs) in response to growth factors using
    BrdU incorporation. Addition of 50 ng/mL PDGF increased PSC
    proliferation to 4.37 +/- 0.49-fold of control, indicating that PDGF
    acts as a strong mitogen for activated PSCs. We interpret the
    50 ng/mL PDGF condition as approximating a near-maximal PDGF stimulus
    and use the reported fold-change over ~24 hours as a quantitative
    constraint on the maximal PDGF-driven proliferation rate parameter
    k_apsc_prolif.

key_assumptions:
    - |
        50 ng/mL PDGF corresponds to a near-saturating concentration for

```

```

    PSC proliferation, so  $PDGF/(PDGF_{50\_prolif} + PDGF) \sim 1$ .
- |
    The proliferation assay duration is approximately 24 hours,
    consistent with typical cytokine incubation protocols.
- |
    BrdU incorporation fold-change is proportional to fold-change in
    PSC cell number (negligible death over 24 hours).
- |
    Rat activated PSC proliferation kinetics in vitro provide an upper
    bound for human tumor aPSC proliferation in vivo.

key_study_limitations:
- |
    Proliferation was measured via BrdU incorporation rather than
    direct cell counts.
- |
    Single PDGF concentration and single assay interval constrain
    only the maximal rate.
- |
    Data from rat PSCs in vitro, not human PDAC-associated stellate cells.
- |
    Background serum proliferative signals not explicitly modeled.

# =====
# DATA SOURCES
# =====
primary_data_source:
doi: "10.1152/ajpcell.2001.281.2.C532"
title: "Identification of mediators stimulating proliferation and
       matrix synthesis of rat pancreatic stellate cells"
authors: [Schneider, Schmid, Liptay, Schmid, Pohl]
year: 2001
source_tag: Schneider2001_PSC_PDGF

secondary_data_sources:
- doi: "10.1136/gut.44.4.534"
  title: "Pancreatic stellate cells are activated by proinflammatory
         cytokines: implications for pancreatic fibrogenesis"
  authors: [Apte, Haber, Darby, Rodgers, McCaughan, Korsten, Pirola, Wilson]
  year: 1999
  source_tag: Apte1999_PSC_Cytokines
  contribution: |
    Provides context on PSC activation and cytokine response protocols.

```

1.3 Schema Structure Summary

The SubmodelTarget schema has the following top-level structure:

- **target_id**: Unique identifier for the calibration target
- **inputs**: List of values extracted from literature with provenance
- **calibration**: Everything needed for inference code generation
 - **parameters**: Parameters to estimate with priors
 - **state_variables**: ODE state variables with initial conditions
 - **model**: Model type specification (built-in or custom)

- `independent_variable`: Time or other independent variable
- `measurements`: Observables with likelihoods
- `identifiability_notes`: Discussion of parameter identifiability
- **experimental_context**: Species, system, cell types, culture conditions
- **study_interpretation**: Scientific interpretation of the data
- **key_assumptions**: Assumptions made in using this data
- **key_study_limitations**: Known limitations of the source study
- **primary_data_source**: Primary literature reference with DOI
- **secondary_data_sources**: Additional references

2 S2: Supported Model Types

The schema supports the following built-in model types:

Table 1: Built-in model types with their ODE formulations.

Model Type	ODE	Parameters
<code>exponential_growth</code>	$dy/dt = k \cdot y$	<code>rate_constant</code>
<code>first_order_decay</code>	$dy/dt = -k \cdot y$	<code>rate_constant</code>
<code>logistic</code>	$dy/dt = k \cdot y \cdot (1 - y/K)$	<code>rate_constant</code> , <code>carrying_capacity</code>
<code>saturation</code>	$dy/dt = k \cdot (1 - y)$	<code>rate_constant</code>
<code>two_state</code>	$dA/dt = -k \cdot A$, $dB/dt = k \cdot A$	<code>forward_rate</code>
<code>michaelis_menten</code>	$dy/dt = -V_{max} \cdot y / (K_m + y)$	<code>vmax</code> , <code>km</code>
<code>direct_conversion</code>	(analytical formula)	<code>formula</code>
<code>direct_fit</code>	(curve fitting)	<code>curve</code>
<code>custom</code>	(user-provided)	<code>code</code> , <code>code_julia</code>

3 S3: Validation Checks

The validation framework includes ten automated checks implemented as Pydantic `@model_validator` methods. Each validator runs after field parsing and raises a `ValidationError` with detailed messages when constraints are violated.

3.1 Summary of Validators

1. **Input reference validation**: All `uses_inputs` references match defined input names
2. **Source reference validation**: All `source_ref` values match defined source tags
3. **Parameter reference validation**: Model parameter roles reference defined parameters
4. **State variable reference validation**: Observable `state_variables` match defined state variables

5. **DOI resolution:** All DOIs resolve via CrossRef API
6. **Value-in-snippet validation:** Extracted values appear verbatim in quoted snippets
7. **Unit validation:** All unit strings parse with Pint
8. **Code syntax validation:** Custom code passes Python AST parsing
9. **Span ordering validation:** Time spans have start < end
10. **ODE model requirements:** ODE models have required state variables and spans

3.2 DOI Resolution Validator

The DOI validator queries the CrossRef API to verify that every cited DOI resolves to a valid publication. This catches fabricated citations before they enter the calibration pipeline. The validator also performs fuzzy matching on title and year to detect metadata inconsistencies.

```
def resolve_doi(doi: str) -> Optional[dict]:
    """Resolve DOI and get metadata from CrossRef."""
    doi_clean = doi.replace("https://doi.org/", "").strip()

    url = f"https://doi.org/{doi_clean}"
    headers = {"Accept": "application/vnd.citationstyles.csl+json"}
    response = requests.get(url, headers=headers, timeout=10)

    if response.status_code != 200:
        return None # DOI does not resolve

    metadata = response.json()
    return {
        "title": metadata.get("title", ""),
        "first_author": metadata.get("author", [{}])[0].get("family", ""),
        "year": metadata.get("issued", {}).get("date-parts", [])[0][0]
    }

@model_validator(mode="after")
def validate_doi_resolution_and_metadata(self) -> "SubmodelTarget":
    """Validate that DOIs resolve and metadata matches."""
    errors = []

    metadata = resolve_doi(self.primary_data_source.doi)
    if metadata is None:
        errors.append(f"DOI '{self.primary_data_source.doi}' failed to resolve")
    else:
        # Check title match (fuzzy)
        if not fuzzy_match(self.primary_data_source.title, metadata["title"]):
            errors.append(f"Title mismatch with CrossRef")
        # Check year match
        if self.primary_data_source.year != metadata["year"]:
            errors.append(f"Year mismatch: {self.primary_data_source.year} vs {metadata['year']}")

    if errors:
        raise ValueError("DOI validation errors:\n - " + "\n - ".join(errors))
    return self
```

Example failure: If an LLM fabricates a citation with DOI 10.1234/fake.2023.12345, the CrossRef query returns a 404 error and validation fails with:

ValidationError: DOI '10.1234/fake.2023.12345' failed to resolve.
Verify at <https://doi.org/10.1234/fake.2023.12345>

3.3 Value-in-Snippet Validator

The value-in-snippet validator checks that each extracted numeric value appears in its associated source text. This catches hallucinations where an LLM extracts a plausible but incorrect value. The validator handles multiple numeric formats including scientific notation, percentages, and Unicode superscripts.

```
def check_value_in_text(text: str, value: float) -> bool:
    """Check if numeric value appears in text."""
    text_norm = text.lower().replace(" ", "")
    patterns = []

    # Direct value and common formats
    patterns.append(str(value))
    patterns.append(f"{value:.1f}")
    patterns.append(f"{value:.2f}")

    # Scientific notation (e.g., "1.5e-3", "1.5E-03")
    if abs(value) < 0.01 or abs(value) > 1000:
        patterns.append(f"{value:e}")
        patterns.append(f"{value:.2e}")

    # Percentage format (if value is between 0 and 1)
    if 0 < value < 1:
        pct = value * 100
        patterns.extend([f"{pct}%", f"{pct:.0f}%", f"{pct:.1f}%"])

    return any(p.lower() in text_norm for p in patterns)

@model_validator(mode="after")
def validate_input_values_in_snippets(self) -> "SubmodelTarget":
    """Validate that extracted values appear in their value_snippet."""
    errors = []
    for inp in self.inputs:
        if not inp.value_snippet:
            continue
        # Skip types that may not have explicit values in text
        if inp.input_type in (InputType.INFERRED_ESTIMATE, InputType.ASSUMED_VALUE):
            continue

        if not check_value_in_text(inp.value_snippet, inp.value):
            errors.append(
                f"Input '{inp.name}': value {inp.value} not found in snippet "
                f"'{inp.value_snippet[:60]}...'")
    )

    if errors:
        raise ValueError("Value-snippet mismatches (possible hallucination):\n - "
                        + "\n - ".join(errors))

    return self
```

Example failure: If an input claims value: 4.37 but the snippet reads “Cell proliferation was 3.21-fold of control”, validation fails with:

```
ValidationError: Value-snippet mismatches (possible hallucination):
- Input 'pdgf_fold_mean': value 4.37 not found in snippet
  'Cell proliferation was 3.21-fold of control...'
```

This validator is particularly effective because LLMs that hallucinate values rarely fabricate consistent snippets—the mismatch provides a reliable detection signal.

3.4 Unit Validation

The unit validator parses all unit strings using Pint, a Python library for physical quantities. Malformed or unrecognized units trigger validation failure.

```
@model_validator(mode="after")
def validate_units_are_valid_pint(self) -> "SubmodelTarget":
    """Validate that all unit strings are valid Pint units."""
    from pint import UnitRegistry
    ureg = UnitRegistry()
    errors = []

    def check_unit(unit_str: str, location: str):
        try:
            ureg(unit_str)
        except Exception as e:
            errors.append(f"{location}: '{unit_str}' is not valid ({e})")

    for inp in self.inputs:
        check_unit(inp.units, f"Input '{inp.name}'")
    for param in self.calibration.parameters:
        check_unit(param.units, f"Parameter '{param.name}'")

    if errors:
        raise ValueError("Invalid Pint units:\n - " + "\n - ".join(errors))
    return self
```

Example failure: If a parameter specifies units: `cells/mL/day` (invalid compound unit), validation fails with:

```
ValidationError: Invalid Pint units:
- Parameter 'k_recruit': 'cells/mL/day' is not valid (...)
```

3.5 Reference Consistency Validators

Four validators check that all cross-references within the schema are consistent:

- **Input references:** Every `uses_inputs` entry and `initial_condition.input_ref` must match a defined input name.
- **Source references:** Every `source_ref` must match `primary_data_source.source_tag` or a `secondary_data_sources[] .source_tag`.
- **Parameter references:** When a model field like `rate_constant` contains a string (not an `InputRef`), it must match a parameter name in `calibration.parameters`.
- **State variable references:** Every state variable referenced in `observable.state_variables` must exist in `calibration.state_variables`.

These validators catch internal inconsistencies where an LLM generates references to entities that don't exist elsewhere in the document.

4 S4: Generated Julia Code Structure

The translator generates Julia scripts with the following structure:

```

using DifferentialEquations
using Turing
using Distributions

# Observed data (from inputs layer)
const obs_proliferation_median = 4.37
const obs_proliferation_sigma = 0.25 # derived from CI95

# ODE function
function ode_proliferation!(du, u, p, t)
    k = p[1]
    du[1] = k * u[1] # exponential growth
end

# Simulate function
function simulate_proliferation(k; t_span=(0.0, 1.0), u0=[1.0])
    prob = ODEProblem(ode_proliferation!, u0, t_span, [k])
    sol = solve(prob, Tsit5())
    return sol[end][1] # final value
end

# Turing model
@model function proliferation_model()
    # Prior
    k_apsc_prolif ~ LogNormal(0.0, 1.0)

    # Simulate
    pred = simulate_proliferation(k_apsc_prolif)

    # Likelihood
    obs_proliferation_median ~ LogNormal(log(pred), obs_proliferation_sigma)
end

# Run inference
model = proliferation_model()
chain = sample(model, NUTS(), MCMCThreads(), 1000, 4)

```

For joint inference over multiple targets, the translator identifies shared parameters by name and generates a single model with one prior per unique parameter.

5 S5: Pydantic Model Definitions

This section provides the key Pydantic model definitions that implement the SubmodelTarget schema. The full implementation is available in the repository at `src/qsp_llm_workflows/core/calibration/sub`

5.1 Top-Level Model

The SubmodelTarget class is the entry point for validation. When a YAML file is loaded, Pydantic recursively validates all nested models.

```

class SubmodelTarget(BaseModel):
    """
    Submodel-based calibration target for QSP parameter inference.

    Separates:
    """

```

```

- `inputs`: What was extracted from papers (with full provenance)
- `calibration`: How to use those inputs for inference
"""
target_id: str = Field(description="Unique identifier for this target")

# Extracted data with provenance
inputs: List[Input] = Field(
    description="Values extracted from literature with full provenance"
)

# Calibration specification
calibration: Calibration = Field(
    description="Everything needed for inference code generation"
)

# Experimental context
experimental_context: ExperimentalContext = Field(
    description="Experimental context for the target"
)

# Study interpretation
study_interpretation: str = Field(
    description="Scientific interpretation of how the study data informs the model"
)
key_assumptions: List[str] = Field(
    description="Key assumptions made in using this data"
)
key_study_limitations: Optional[List[str]] = Field(
    default=None, description="Known limitations of the study"
)

# Data sources
primary_data_source: PrimaryDataSource = Field(
    description="Primary literature source"
)
secondary_data_sources: Optional[List[SecondaryDataSource]] = Field(
    default=None, description="Additional literature sources"
)

# Validators (10 total) defined via Pydantic decorators
# See Section S3 for implementation details

```

5.2 Input Model

Each input captures a single value extracted from literature with full provenance:

```

class InputType(str, Enum):
    """Type of input extracted from literature."""
    DIRECT_MEASUREMENT = "direct_measurement" # Value reported directly
    PROXY_MEASUREMENT = "proxy_measurement" # Requires conversion
    EXPERIMENTAL_CONDITION = "experimental_condition" # Protocol choice
    INFERRED_ESTIMATE = "inferred_estimate" # Interpreted from qualitative text
    ASSUMED_VALUE = "assumed_value" # Domain knowledge, not in source

class InputRole(str, Enum):
    """Role of input in calibration."""
    INITIAL_CONDITION = "initial_condition" # IC for ODE integration
    TARGET = "target" # Calibration target (likelihood term)
    FIXED_PARAMETER = "fixed_parameter" # Fixed value in model
    AUXILIARY = "auxiliary" # Supporting data

class Input(BaseModel):
    """A value extracted from literature with full provenance."""
    name: str = Field(description="Unique identifier for this input")
    value: float = Field(description="Extracted numeric value")
    units: str = Field(description="Units of the value")
    uncertainty: Optional[Uncertainty] = Field(default=None)

```

```

n: Optional[int] = Field(default=None, description="Sample size")
input_type: InputType
role: Optional[InputRole] = Field(default=None)
source_ref: str = Field(description="Reference to source_tag in data sources")
source_location: str = Field(description="Location within the source")
value_snippet: Optional[str] = Field(
    default=None,
    description="Exact text from paper containing the value (for validation)"
)

```

5.3 Model Type Discriminated Union

The schema uses Pydantic's discriminated union pattern to support multiple model types, each with type-specific required fields:

```

class BaseModelSpec(BaseModel):
    """Base class for all model specifications."""
    data_rationale: str = Field(
        description="Why this model type fits the experimental data"
    )
    submodel_rationale: str = Field(
        description="Why this is a valid submodel of the full QSP model"
    )

class ExponentialGrowthModel(BaseModelSpec):
    """Exponential growth:  $dy/dt = k * y$ """
    type: Literal["exponential_growth"] = "exponential_growth"
    rate_constant: ParameterRole = Field(
        description="Rate constant parameter name or input_ref"
    )

class TwoStateModel(BaseModelSpec):
    """Two-state transition:  $A \rightarrow B$  with first-order kinetics."""
    type: Literal["two_state"] = "two_state"
    forward_rate: ParameterRole = Field(
        description="Forward transition rate constant"
    )

class CustomModel(BaseModelSpec):
    """Custom ODE with user-provided code"""
    type: Literal["custom"] = "custom"
    code: str = Field(description="Python ODE function")
    code_julia: str = Field(description="Julia ODE function for inference")

# Discriminated union selects model class based on 'type' field
Model = Annotated[
    Union[
        FirstOrderDecayModel,
        ExponentialGrowthModel,
        LogisticModel,
        MichaelisMentenModel,
        TwoStateModel,
        SaturationModel,
        DirectConversionModel,
        DirectFitModel,
        CustomModel,
    ],
    Field(discriminator="type"),
]

```

The `discriminator="type"` annotation tells Pydantic to inspect the `type` field in the YAML to determine which model class to instantiate. This ensures that an `exponential_growth` model is validated against `ExponentialGrowthModel` (which requires `rate_constant`) while a `two_state` model is validated against `TwoStateModel` (which requires `forward_rate`).

5.4 Prior Specification

Priors are specified with distribution type and parameters:

```
class PriorDistribution(Enum):
    """Supported prior distribution types."""
    LOGNORMAL = "lognormal" # For positive parameters (rates, densities)
    NORMAL = "normal" # For unconstrained parameters
    UNIFORM = "uniform" # For bounded parameters
    HALF_NORMAL = "half_normal" # For positive parameters with mode at 0

class Prior(BaseModel):
    """Prior distribution specification for Bayesian inference."""
    distribution: PriorDistribution
    mu: Optional[float] = Field(default=None, description="Location parameter")
    sigma: Optional[float] = Field(default=None, description="Scale parameter")
    lower: Optional[float] = Field(default=None, description="Lower bound")
    upper: Optional[float] = Field(default=None, description="Upper bound")
    rationale: Optional[str] = Field(default=None)

    @model_validator(mode="after")
    def validate_prior_params(self) -> "Prior":
        """Validate that required parameters are provided for each distribution."""
        if self.distribution == PriorDistribution.LOGNORMAL:
            if self.mu is None or self.sigma is None:
                raise ValueError("lognormal prior requires mu and sigma")
        elif self.distribution == PriorDistribution.UNIFORM:
            if self.lower is None or self.upper is None:
                raise ValueError("uniform prior requires lower and upper")
            if self.lower >= self.upper:
                raise ValueError("uniform lower must be < upper")
        return self
```

5.5 Loading and Validating YAML Files

To load and validate a calibration target:

```
import yaml
from qsp_llm_workflows.core.calibration import SubmodelTarget

# Load YAML file
with open("psc_proliferation_PDAC_deriv001.yaml") as f:
    data = yaml.safe_load(f)

# Parse and validate (raises ValidationError on failure)
target = SubmodelTarget(**data)

# Access validated fields
print(f"Target: {target.target_id}")
print(f"Parameters: {[p.name for p in target.calibration.parameters]}")
print(f"Model type: {target.calibration.model.type}")
```

If validation fails, Pydantic raises a `ValidationError` with detailed error messages:

```
pydantic_core._pydantic_core.ValidationError: 2 validation errors for SubmodelTarget
calibration.measurements.0.uses_inputs.0
  Input 'typo_input_name' not found in inputs [...]
primary_data_source.doi
  DOI '10.1234/fake' failed to resolve [...]
```